WanderWise – AI-Powered Travel Itinerary Planner

Date	23 October 2026
Team ID	0827CY231029
Project Name	WanderWise – AI Travel Itinerary Planner

Project Description

WanderWise is a personalized travel itinerary planner for solo travelers, couples and small groups. It combines ready-made trips with an AI-assisted builder that creates a dated, day-by-day schedule from destination, budget, style and preferences.

The React interface presents saved trips and daily activities. The FastAPI backend handles authentication, input validation, planning and persistence through Supabase or SQLite. Optional AI providers enhance generation; a local rule-based engine supports operation without API keys.

Scenarios

- Scenario 1: A solo traveler opens the Goa template, reviews its daily plan and saves a dated copy to My trips.

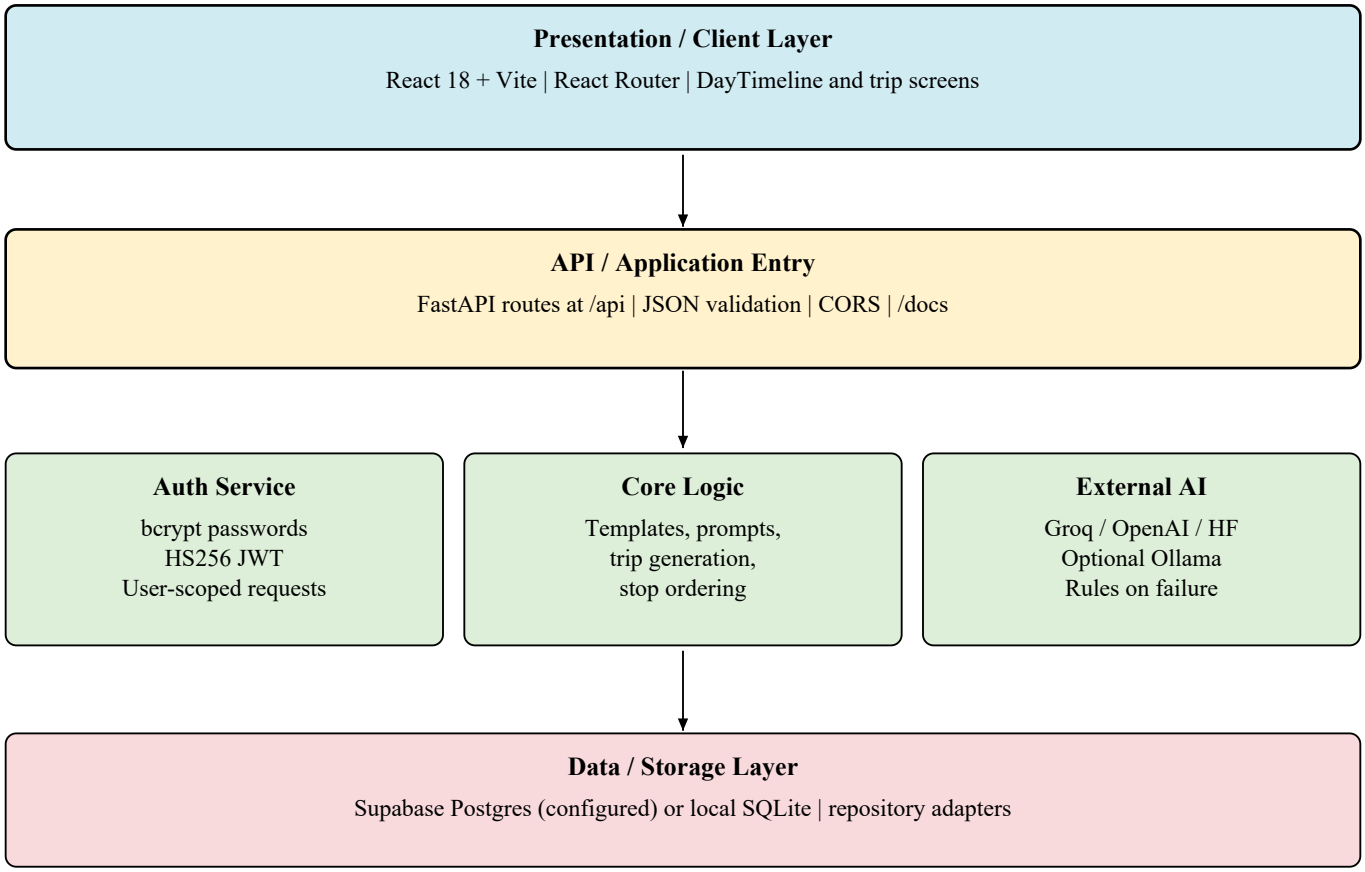
- Scenario 2: A couple enters a destination, dates, budget, romance/culture style and pace. WanderWise generates a plan they can review and edit.

- Scenario 3: A traveler planning with companions shares a public itinerary link and downloads an ICS file for calendar use.

These scenarios illustrate intended flows; they do not represent recorded user research or a newly executed live demo.

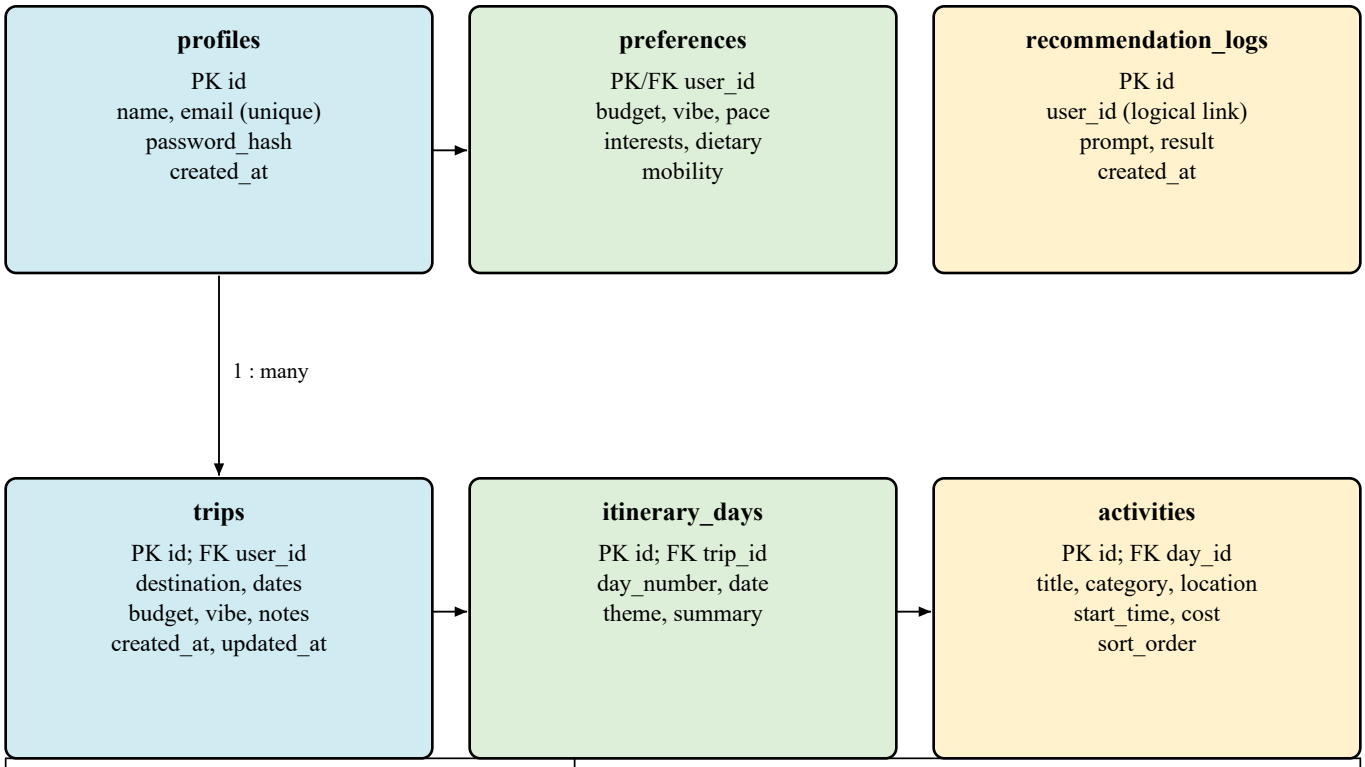
Technical Architecture

The browser calls a single modular API service over JSON. Authentication, trip routes, prompt generation and data access remain separate modules within the FastAPI application.



Automatic provider selection chooses one configured provider. A failed or absent provider falls back to local rules. The production database mode is selected from Supabase configuration at startup.

Database ER Diagram and Pre-requisites



Relationship	Cardinality / Notes
profiles -> preferences	One profile has one saved preference record, linked by user_id.
profiles -> trips	One profile can have many trips.
trips -> itinerary_days	One trip can have many dated days.
itinerary_days -> activities	One day can have many ordered activities.
profiles -> recommendation_logs	Logical user association; SQLite schema does not declare this FK.

Pre-requisites: Python 3.11+, Node.js 18+, a browser and an editor. Optional: Supabase account, hosted AI credentials or a local Ollama model.

Project Workflow

Milestone	Activities and Deliverables
1. Model Selection and Architecture	Choose provider mode, prepare environment variables, define API/data flow and inspect data schema.
2. Core Functionalities Development	Implement templates, preferences, trip generation, recommendations and persistence.
3. Main Application Development	Wire FastAPI routers, validation, errors, AI prompts and rule-based fallback.
4. Frontend Development	Build discovery, auth, preferences, builder, saved-trip, timeline and share screens.
5. Deployment	Install dependencies, run locally, configure Render/Vercel/Supabase and validate flows.
6. Conclusion	Review delivered features, practical limitations and future roadmap.

The source is already organized around these responsibilities. The workflow describes the project structure and intended validation sequence.

Milestone 1: Model Selection and Architecture

Activity 1.1: Configure Optional AI Access

- Create provider credentials in the selected service account. Store values only in backend/.env or the deployment environment.

- Set GROQ_API_KEY for the default live provider, OPENAI_API_KEY for OpenAI, or HF_TOKEN for Hugging Face.

- For local Ollama, select LLM_PROVIDER=ollama and configure OLLAMA_URL/OLLAMA_MODEL.

- Do not require an AI key for the baseline local demo: the rules engine can create structured day plans.

Configuration Reference

Setting	Project Behavior
LLM_PROVIDER=auto	Select Groq, then OpenAI, then HF based on available credentials.
LLM_PROVIDER=ollama	Select the local Ollama integration explicitly.
Provider request fails	Generation returns to the local rules engine.

Project sources: backend/app/config.py; services/llm/__init__.py; services/ai_engine.py.

Milestone 1: Select the Generative Model

Activity 1.2: Model Defaults and Selection

Provider	Default Model in Source	Configuration
Groq	llama-3.1-8b-instant	GROQ_MODEL
OpenAI	gpt-4o-mini	OPENAI_MODEL
Hugging Face	mistralai/Mistral-7B-Instruct-v0.2	HF_MODEL
Ollama	llama3	OLLAMA_MODEL

Defaults are configurable and may need adjustment if a provider changes its supported models. Model choice should balance output structure, latency and service availability.

Selection Criteria

- Return valid JSON day plans with timed activities and usable labels.

- Follow the destination, dates, style, pace and practical notes supplied by the traveler.

- Measure quality and latency on sample destinations before choosing production settings.

No comparative model benchmark is available in the supplied files.

Milestone 1: Architecture and Development Environment

Activity 1.3: Define Application Interactions

- React screens call /api routes through the shared frontend API helper.

- FastAPI validates Pydantic inputs and enforces authentication on private operations.

- AI service modules build prompts and return structured days; repository adapters persist the plan.

Activity 1.4: Prepare the Environment

Layer	Required Files / Tools
Backend	Python; requirements.txt; .env.example; app/main.py.
Frontend	Node.js; package.json; vite.config.js; src/App.jsx.
Local database	database/schema.sqlite.sql; automatically initialized when Supabase is absent.
Hosted database	Supabase project; database/schema.sql; migration script if upgrading.

Project sources: 5. Project Development Phase/; 7. Project Documentation/SETUP.md.

Milestone 2: Core Functionalities Development

Activity 2.1: Implement the Core Planner

Feature	Behavior
Authentication	Register and log in with email/password; bcrypt hashes and JWT.
Traveler preferences	Save style, pace, interests, dietary and mobility notes.
Ready-made itineraries	Browse six curated destinations and clone a trip to saved plans.
Custom AI trip builder	Generate dated day plans from destination, budget and travel inputs.
Trip management	List, rename and delete trips; edit activities; request regeneration/refinement.
Public sharing	Generate a unique link so a viewer can read a shared itinerary without login.
Multi-format export	Download JSON from the API; Markdown and ICS from the trip screen.
Destination recommendations	Rank destinations from preferences using AI or catalog scoring.

Planning Output

A trip contains dated day entries. Each day has a theme, summary and ordered activities containing titles, time slots, locations and estimated costs. Local logistics add accommodation and transport suggestions.

The app is a planning aid; it does not book accommodation, make payments or verify current venue availability.

Milestone 2: Implement the FastAPI Backend

Activity 2.2: Routes, Input Processing and Persistence

Module	Purpose
routers/auth.py	Register/login and return a token response.
routers/preferences.py	Store preferences and produce destination recommendations.
routers/templates.py	Browse templates, show details and clone a template.
routers/trips.py	Create, fetch, update, delete, generate, refine, share and export trips.
store.py / repository adapters	Select Supabase or local repository operations.
services/ai_engine.py	Coordinate model output, local rules, ranking and logistics.

Processing Sequence

- Validate the request, derive travel style and normalize date values.
- Read relevant saved preferences, create a plan and persist trip/day/activity records.
- Return a structured response and log generation inputs where the route supports it.

Project sources: backend/app/routers/ and services/ai_engine.py.

Milestone 3: Main Application Development

Activity 3.1: Main Entry Point and Core Routes

Endpoint / Group	Purpose
GET /api/health	Service status, selected database mode and AI provider.
/api/auth	Registration/login and account authentication.
/api/preferences	Preference storage and recommendation requests.
/api/templates	Template listing, details and cloning.
/api/trips	Authenticated trip creation and management.
GET /api/share/{token}	Public read-only view of a token-linked itinerary.

Validation and Error Responses

- Trip end date must be on or after the start date.

- Budget is non-negative; travelers are 1 to 20; pace/style use enumerated values.

- Invalid credentials produce 401; absent records produce 404; validation errors produce 422.

Project sources: backend/app/main.py, schemas.py and security.py.

Milestone 3: Prompt Engineering Helpers

Build Structured Itinerary and Recommendation Prompts

prompt_builder.py creates constrained generation prompts from destination, date range, travel style, budget, pace, travelers, interests and practical notes.

Prompt Input	Purpose
Destination and date range	Define trip location and the expected number of dated days.
Travel style and interests	Guide themes, attractions and dining suggestions.
Budget and travelers	Inform cost-aware planning and group-size context.
Pace, dietary and mobility notes	Guide schedule intensity and relevant descriptions.
Feedback and requested day	Ask for a revised plan; exact unchanged-day preservation needs validation.

Output Structure

The application expects a day list with activity details. Structured JSON lets the timeline, persistence layer and export formats consume the same plan.

Project sources: backend/app/services/prompt_builder.py; services/ai_engine.py; routers/trips.py.

Milestone 3: AI Output and Local Fallback

Integrate Responses and Keep Baseline Planning Available

- complete_json() asks the selected provider for JSON and returns no result if the provider is absent or fails.

- generate_itinerary() uses model days when available; otherwise local_itinerary() assembles a plan from the destination catalog.

- Local selection ranks activity candidates by travel style and interests, applies time slots and adds logistics.

- Destination recommendations use model output or locally scored catalog destinations.

Important Implementation Limits

The provider integration is optional. After a failed request, the current engine returns to rules instead of trying all other providers. Local planning uses estimates and does not validate real-time venue hours, prices or accessibility.

Per-day refinement adds a day request to the prompt, while the endpoint replaces the returned plan. The local fallback may ignore refinement feedback.

Project sources: backend/app/services/llm/; services/ai_engine.py; routers/trips.py.

Milestone 4: Frontend Development

Activity 4.1: Design and Develop the User Interface

Page / Component	Role
Home and Templates	Destination discovery and ready-made itinerary cards.
Login and Register	Authentication and account creation.
Preferences and Recommend	Travel preferences and destination ideas.
TripBuilder	Destination, dates, budget, style, travelers and practical notes.
Dashboard and TripDetail	Saved trips, daily timelines and plan actions.
ShareView	Public read-only itinerary screen.
DayTimeline and TemplateCard	Reusable timeline and curated-trip components.

React Router provides public and private routes. CSS handles responsive layouts; Framer Motion supplies animations and Lucide supplies icons.

Project sources: frontend/src/App.jsx, pages/, components/ and index.css.

Milestone 4: Dynamic Rendering and User Actions

Activity 4.2: Connect React Screens to the API

- The authentication context stores the token and supplies signed-in state to route guards.
- The shared API client prefixes /api and sends authenticated requests when needed.
- TripBuilder collects traveler inputs and renders the returned saved trip.
- TripDetail exposes regeneration/refinement, activity changes, public sharing and downloads.
- Export controls download Markdown and ICS; the backend also supports JSON export.

User Feedback

Use loading states while requests run, readable API errors when inputs fail and a refreshed timeline after successful changes. The final verification should cover empty states, invalid dates and failed generation requests.

Project sources: frontend/src/api/; pages/TripBuilder.jsx and TripDetail.jsx; components/DayTimeline.jsx.

Milestone 5: Local Deployment

Activity 5.1: Prepare Local Dependencies

Use the current source folders inside 5. Project Development Phase. Open separate terminals for backend and frontend.

```
cd "5. Project Development Phase/backend"
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
```

The activation and copy commands above are for Windows. Python 3.11+ and Node.js 18+ are the documented prerequisites.

Environment Preparation

- Choose a strong JWT signing secret and set it in backend/.env.

- Keep optional provider and Supabase keys empty for a local SQLite/rules baseline.

- When enabling Supabase, create the database tables using the provided schema first.

Project sources: 7. Project Documentation/SETUP.md; backend/app/config.py.

Milestone 5: Backend Configuration and Startup

Configure and Run the API

Setting	Use
SECRET_KEY / JWT_SECRET	JWT signing secret; replace the development default.
FRONTEND_URL / CLIENT_ORIGIN	Frontend origin, normally http://localhost:5173 locally.
SUPABASE_URL / SUPABASE_SERVICE_ROLE_KEY	Enable hosted Postgres mode when both are configured.
GROQ_API_KEY / OPENAI_API_KEY / HF_TOKEN	Optional hosted generation credentials.
LLM_PROVIDER	Automatic provider selection or an explicitly selected integration.

```
uvicorn app.main:app --reload --port 5000
```

Open http://localhost:5000/api/health and confirm status is ok. The response identifies sqlite/supabase and the selected model provider or rules. Interactive API documentation is at /docs.

Project sources: backend/app/config.py and main.py; 7. Project Documentation/SETUP.md.

Milestone 5: Frontend Startup and Local Verification

```
cd "5. Project Development Phase/frontend"
npm install
npm run dev
```

Open http://localhost:5173. Vite proxies /api requests to the backend on port 5000.

Activity 5.2: Verification Sequence

- Register and log in; confirm the signed-in navigation.

- Browse a curated template, save it and open its daily timeline.

- Generate a custom trip; check dates, labels, costs and activity order.

- Change an activity, request refinement and review all returned days.

- Open a share link in a separate browser session; export Markdown and ICS.

The existing testing README reports 10 manual cases as passed. This documentation work did not repeat those runtime tests. Project sources: frontend/vite.config.js; 6. Project Testing/README.md.

Milestone 5: Production Deployment

Activity 5.3: Render, Vercel and Supabase

Service	Current Folder / Configuration	Deployment Action
Supabase	backend/database/schema.sql	Create schema and apply the trips migration if upgrading.
Render API	5. Project Development Phase/backend	Install requirements; use Gunicorn with Uvicorn worker; set server secrets and frontend origin.
Vercel UI	5. Project Development Phase/frontend	Run npm install and npm run build; publish dist; set VITE_API_URL.

Existing deployment guides describe older root paths named backend/frontend. Use the phase-folder paths in the current repository layout and adjust render.yaml root settings if needed.

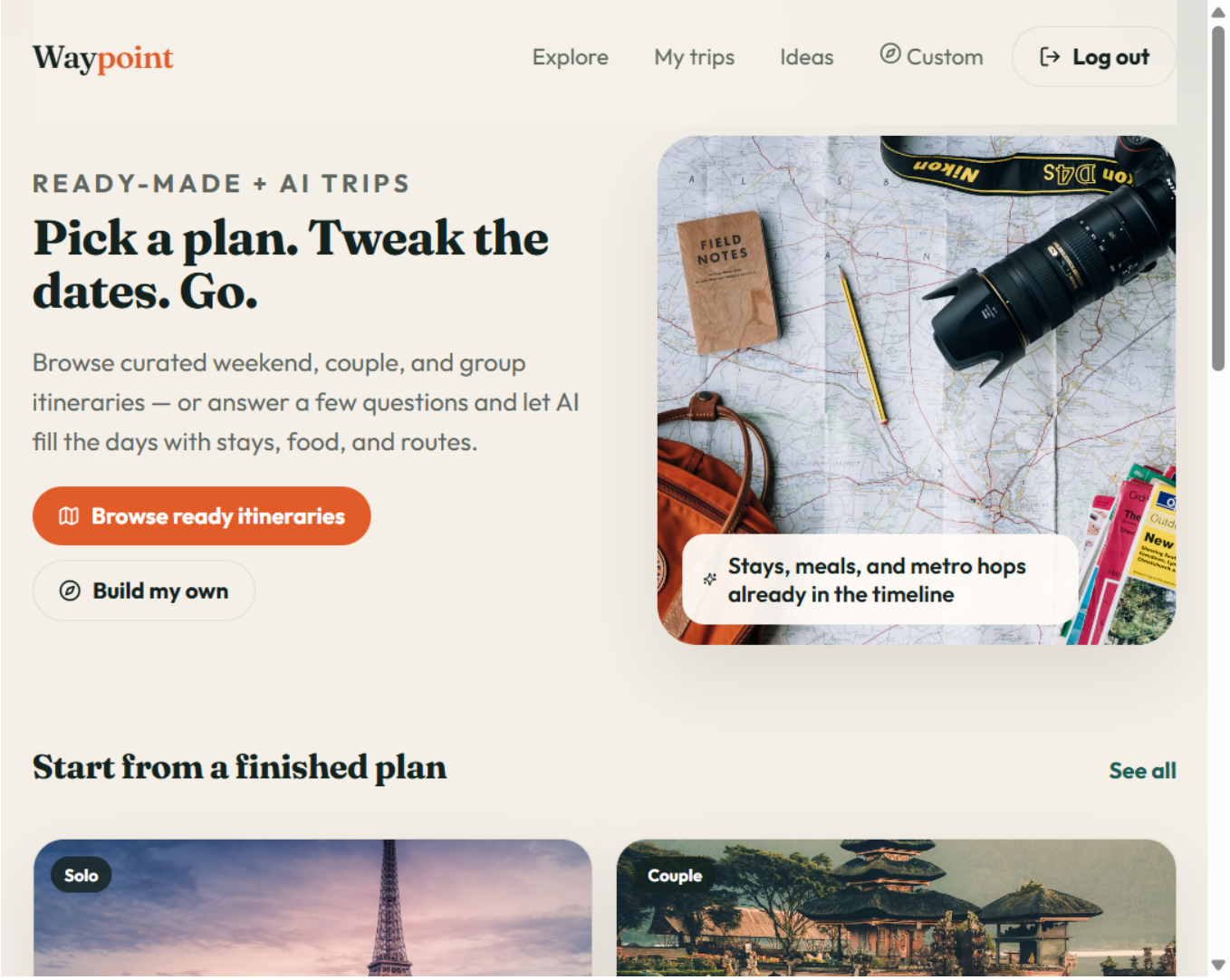
After deployment, set the frontend origin in the backend, verify /api/health, test registration/generation and check public share URLs. Public deployment URLs are not supplied in the project files.

Project sources: 7. Project Documentation/DEPLOYMENT.md; render.yaml; frontend/vercel.json.

Exploring the Website's Web Pages

Home / Discovery Page

The home page highlights ready-made and AI trips. Navigation leads to Explore, saved trips, Ideas and the Custom builder.



Existing project preview supplied in 8. Project Demonstration/preview.png; not a screenshot captured during this documentation task.

Input Section and Results Section

Custom Trip Builder

The traveler provides a destination, start/end dates, number of travelers, budget, travel style, interests, pace and optional dietary/mobility notes. Validation rejects an inverted date range and invalid values.

Daily Timeline and Saved Trip

The result is a saved trip with dated day entries, themes and timed activities. The trip view includes estimated spending, regeneration/refinement, activity edits, share-link creation and export controls.

Template and Recommendation Flows

A curated template can be cloned into the same saved-trip workflow. The Ideas page ranks destinations using travel preferences.

Practical Review

- Verify suggested places, availability, transfer assumptions and prices before travel.
- Review all days after refinement because unchanged-day preservation is not guaranteed by the current local fallback.

Project sources: frontend/src/pages/TripBuilder.jsx, TripDetail.jsx, Templates.jsx and Recommend.jsx.

Conclusion and Future Scope

WanderWise demonstrates an integrated travel-planning workflow: discover a destination, choose a curated trip or generate a personal plan, review a daily schedule, save changes, and share or export the itinerary.

The combination of React, FastAPI and repository adapters keeps the interface, planning logic and storage concerns separate. Optional hosted AI improves generation while local rules support baseline operation.

Current Limits

- Destination catalog coverage and model suggestions are finite and require traveler review.
- Cost and transfer estimates are not verified live travel data.
- No recorded public demo, performance benchmark or production uptime report is supplied.

Future Scope

- Multi-city trip planning, simultaneous trip editing, offline access and a native mobile app.
- Better unchanged-day refinement, measured load capacity and stronger location/time validation.

Project sources: README.md; phase READMEs; backend/app/; frontend/src/; documentation guides.